

A Serverless Architecture and MLOps Governance for Dynamic Calibration of Sensor Networks in Forest Microclimates

dos Santos AAF^{a,b,c,*,1}

^aUniversity of São Paulo, São Paulo, SP, Brazil

ARTICLE INFO

Keywords:

Serverless Computing
Sensor Calibration
MLOps
AWS Lambda
Amazon DynamoDB
Data Drift
AWS IoT Core
Forest Microclimate.

ABSTRACT

Low-cost wireless sensor networks deployed in forest microclimates suffer severe metrological degradation caused by hygrothermal hysteresis, transient condensation, and the accelerated ageing of capacitive transducers. This paper designs, develops, deploys, and experimentally validates an end-to-end production data engineering and MLOps infrastructure on a public cloud (AWS) for continuous relative humidity calibration. The system implements a heterogeneous ingestion layer (MQTT with QoS 1 delivery guarantees and HTTPS REST batches authenticated via SigV4 in AWS IoT Core), deterministic real-time serverless inference packaged as an OCI image on AWS Lambda, and partitioned persistence in Amazon DynamoDB. The analytical pipeline incorporates strict temporal partitioning without data leakage, continuous covariate shift monitoring via a two-tailed Kolmogorov–Smirnov test, and absolute arithmetic parity auditing ($\Delta < 10^{-4}$) between development and production instances. Benchtop experiments with empirical data (18,550 synchronised readings) demonstrated a substantial reduction in Mean Absolute Error (MAE) from 5.404% to 2.422% Relative Humidity (RH) (a net improvement of 55.19%), constraining residual error dispersion to $\sigma = 1.3720\%$ RH. However, it should be noted that the restricted dynamic range and low seasonal variability observed in the dataset limit the model's ability to learn dynamic responses under extreme weather regimes; thus, it serves as a baseline model validated for the sign-off and operational maintenance of the production pipeline. From a computational perspective, effective per-request latency in a warm state reached 10.95 ms with 232 MB RAM usage, enabling continuous execution at negligible on-demand costs compared with permanently virtualised architectures.

1. Introduction

Monitoring environmental variables in forest microclimates is essential to understand soil water retention, assess wildfire risks, and track the ecological dynamics of vegetation.[6] However, enabling such studies across broad geographical scales presents significant cost and scalability challenges. Primary reference meteorological stations (scientific bench-grade instruments) incur acquisition and maintenance costs that are prohibitive for high-density field deployments. To overcome this financial barrier, a practical alternative is the deployment of wireless sensor networks (WSNs) based on low-cost commercial integrated modules, such as the BME280 (temperature, humidity, and barometric pressure) and the BME680 (which additionally incorporates the detection of volatile organic compounds and air resistance) [14, 5, 10]

Despite their economic appeal, transducers such as the BME280 and BME680 suffer from severe physical limitations when exposed continuously to forest microclimates: pronounced hygrometric hysteresis, cross-thermal perturbations induced by parasitic board heating, and susceptibility to sensor saturation in persistent dew and condensation conditions. The literature commonly addresses these deviations by proposing algorithmic calibration routines developed on benchtop setups [14, 11]. However, most of these solutions remain confined to isolated prototyping scripts (e.g., local exploratory notebooks), failing to address how such predictive intelligence can be operated continuously at scale.

A central bottleneck in enabling high-density ecological and industrial deployments shifts from isolated theoretical modelling to the actual productisation of the sensor calibrator. Deploying a correction algorithm into a production data pipeline requires addressing critical challenges in Data Engineering and Machine Learning Operations (MLOps) [9]:

1. **Heterogeneous Ingestion and Network Reliability:** Remote field devices require support for lightweight, event-driven protocols with delivery guarantees (MQTT with Quality of Service (QoS) 1) [1] whilst the underlying infrastructure must also accommodate batch uploads (HTTPS REST) following communication blackout intervals;
2. **Deterministic Runtime Inference:** Ensuring that the cloud prediction runtime executes the exact mathematical model trained offline, with strict arithmetic parity ($\Delta < 10^{-4}$) and without prohibitive cold-start latencies;
3. **Methodological Protection Against Data Leakage:** Ensuring that the temporal aggregation between field sensors and the reference station is structured under strict temporal partitioning without data leakage [2]
4. **Operational Maintenance and Continuous Monitoring (Covariate Shift):** Establishing mechanisms to detect when seasonal climatic shifts invalidate the production calibrator, signalling the need for automated retraining before downstream data corruption occurs.

To address these demands, this paper designs, implements, and validates an end-to-end Data Engineering and

 ado1phoafsantos@usp.br (d.S. AAF)
ORCID(s): 0000-0002-6225-9195 (d.S. AAF)

MLOps architecture on the Amazon Web Services (AWS) public cloud for the productisation of dynamic low-cost sensor calibration. The workflow is fully structured by parameterising stages ranging from ingestion via AWS IoT Core and containerised inference on AWS Lambda to NoSQL persistence in Amazon DynamoDB and deterministic audit routines.

2. Theoretical Background

2.1. Metrological Degradation in Low-Cost Sensors

Silicon-integrated low-cost transducers, such as the BME280 and BME680 modules, employ hygroscopic polymer layers whose dielectric constant varies as a function of atmospheric water vapour absorption. Although economically advantageous, these devices face severe mechanical and chemical instabilities when operating under persistent condensation in forest microclimates:

1. **Cross-Temperature Interference:** Resistive heating from the printed circuit board (PCB) itself (self-heating) alters the local relative humidity (RH) at the sensing surface;
2. **Adsorption/Desorption Hysteresis:** The time required to release water molecules trapped within the micropores of the polymer exceeds the absorption time, generating divergent trajectories between diurnal and nocturnal cycles;
3. **Cumulative Offset:** The deposition of particulate matter and volatile organic compounds (VOCs) induces permanent degradation of sensor sensitivity.

Mathematically, the reading observed by the field sensor $y_{\text{raw}}(t)$ can be formulated as the superposition of the true environmental signal $H(t)$ and systematic as well as stochastic perturbation terms:

$$y_{\text{raw}}(t) = H(t) + \beta_0 + \beta_1 T(t) + h(t) + \epsilon(t), \quad (1)$$

where β_0 models the zero-shift bias, $\beta_1 T(t)$ denotes thermal interference, $h(t)$ accounts for dynamic hysteresis, and $\epsilon(t)$ represents Gaussian stochastic noise. The purpose of the dynamic calibrator is to estimate $H(t)$ from the imperfect input vector $\mathbf{x}(t) = [y_{\text{raw}}, T, \text{CO}_2]$.

2.2. Decision-Tree-Based Non-Linear Regression

To map the discontinuities caused by condensation and hysteresis, a Decision Tree Regressor is employed [4, 8]. Unlike linear regressors, decision trees do not assume global monotonic dependencies; instead, they recursively partition the multidimensional input space into disjoint hyperrectangles $R_1, R_2, \dots, R_J$.

The prediction $\hat{y}$ associated with a new field reading $\mathbf{x}$ corresponds to the arithmetic mean of the reference benchtop labels (y_{ref}) contained within the terminal region (leaf node) reached:

$$\hat{y}(\mathbf{x}) = \sum_{j=1}^J c_j \cdot \mathbb{I}(\mathbf{x} \in R_j), \quad c_j = \frac{1}{|R_j|} \sum_{\mathbf{x}_i \in R_j} y_{\text{ref},i}, \quad (2)$$

where $\mathbb{I}(\cdot)$ denotes the indicator function.

At each internal node of the tree, the algorithm selects the physical predictor feature k and the split threshold s that maximise the reduction in Mean Squared Error (MSE):

$$\min_{k,s} \left[\sum_{\mathbf{x}_i \in R_1(k,s)} (y_{\text{ref},i} - c_1)^2 + \sum_{\mathbf{x}_i \in R_2(k,s)} (y_{\text{ref},i} - c_2)^2 \right], \quad (3)$$

with $R_1(k, s) = \{\mathbf{x} \mid x_k \leq s\}$ and $R_2(k, s) = \{\mathbf{x} \mid x_k > s\}$.

The primary advantage of this model class for production environments lies in its minimal computational overhead at inference time: algorithmic complexity is bounded by $\mathcal{O}(d_{\text{max}})$ simple conditional comparisons (*if-else*), where d_{max} represents the maximum depth of the tree, thereby eliminating the need for dense matrix algebra during runtime execution.

2.3. Metrological Evaluation Metrics

Quantifying the metrological precision gains and physical adherence of the calibration relies on two complementary metrics. [7]

- **Mean Absolute Error (MAE):**

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_{\text{ref},i} - \hat{y}_i| \quad (4)$$

Measures the average absolute magnitude of deviations in the physical unit of the measured quantity (% RH). It provides a straightforward interpretation and weights all individual errors linearly, offering a clear estimate of the instrument's typical operational error.

- **Root Mean Squared Error (RMSE):**

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_{\text{ref},i} - \hat{y}_i)^2} \quad (5)$$

By squaring residual differences prior to averaging, RMSE penalises large-magnitude isolated errors more severely than MAE. In environmental sensor calibration, substantial divergence between MAE and RMSE indicates isolated saturation or extreme condensation events that remain uncompensated by the model.

2.4. Foundations of Event-Driven Serverless Computing

The serverless paradigm decouples software development from the provisioning and maintenance of permanent dedicated servers [12]. In an event-driven architecture based on Function-as-a-Service (FaaS), computational functions are triggered strictly upon message arrival, executing business logic within ephemeral environments isolated by micro-virtual machines (microVMs), such as AWS Firecracker.

For MLOps pipelines supporting near-edge or edge-ingested inference, the serverless execution model addresses operational cost bottlenecks in WSNs:

1. **Elimination of Idle Costs:** Ecological networks typically sample at discrete temporal intervals (ranging from tens of seconds to several minutes). Dedicated virtual machines allocated continuously operate with low duty cycles, resulting in substantial budgetary waste;
2. **Runtime Lifecycle and Caching:** The initial invocation of a function incurs execution overhead to initialise the microVM and load dependencies (cold start). However, subsequent invocations reuse the execution context preserved in memory (warm start), enabling deterministic inference latencies within tens of milliseconds;
3. **OCI Containerisation:** Encapsulating prediction routines under the Open Container Initiative (OCI) standard ensures the portability of complex numerical artefacts and strict binary compatibility across compiled C/C++ libraries (such as Scikit-Learn and NumPy). [13]

2.5. Governance and Data Drift Detection via Kolmogorov-Smirnov

Machine learning models deployed on ecological time series are inherently vulnerable to covariate shift: incoming meteorological variables undergo statistical distribution changes driven by seasonal climatic transitions (e.g., dry-to-wet seasonal shifts), whereas the internal parameters of the calibrated model remain static.

Detecting such operational degradation in production without access to real-time ground-truth reference labels is carried out using the two-tailed Kolmogorov-Smirnov (KS) test [3]. The test evaluates the Kolmogorov-Smirnov distance (D_{KS}), which quantifies the maximum vertical distance between the empirical cumulative distribution functions (CDFs) of the baseline historical dataset $F_{base}(u)$ and the current production monitoring window $F_{prod}(u)$:

$$D_{KS} = \sup_{u \in \mathbb{R}} |F_{base}(u) - F_{prod}(u)|, \quad (6)$$

where $\sup$ denotes the supremum operator.

When the observed D_{KS} statistic exceeds the critical threshold associated with the designated significance level ($\alpha = 0.05$), the null hypothesis of distributional equivalence is rejected. This outcome indicates that the operational meteorological dynamics have drifted beyond the training envelope, automatically activating retraining triggers within the MLOps pipeline.

2.6. Numerical Parity and Runtime Determinism

In MLOps architectures, numerical parity defines the capability of a serialised artefact to generate identical outputs when evaluated against identical inputs across heterogeneous execution environments (e.g., local development versus cloud inference) [9]. Although theoretically deterministic, models trained in interpreted languages such as Python rely on underlying compiled C/C++ routines and optimised linear algebra libraries (e.g., BLAS, LAPACK, OpenBLAS, or Intel MKL).

Subtle version discrepancies across these low-level dependencies, microarchitectural differences in CPU instruction sets (e.g., AVX-512 vectorisation versus ARM NEON), or platform-specific rounding behaviours under the IEEE 754 floating-point standard can introduce silent prediction drift. In metrological calibration systems, accumulated minor numerical divergences compromise the historical comparability of ecological time series.

To validate the algorithmic neutrality of the infrastructure, an audit criterion is established based on the maximum absolute scalar tolerance:

$$\Delta = |\hat{y}_{cloud} - \hat{y}_{local}| < \tau, \quad (7)$$

where a conservative threshold of $\tau = 10^{-4}\%$ RH is adopted. Containerisation via immutable OCI images plays a pivotal role in this verification process by freezing the base operating system, compiler toolchains, and exact binary versions of the analytical dependencies between the offline training environment and serverless execution.

3. Methodology and Architecture Development

Implementing the dynamic calibration pipeline required the development of a decoupled, scalable, and fault-tolerant infrastructure designed to absorb the heterogeneity of field transmissions and deliver predictions with guaranteed analytical integrity. The pipeline was structured into eight sequential technical stages:

1. **Integrated Solution Topology:** Macro-architectural definition of the cloud infrastructure and data workflows;
2. **Security and Access Control (IAM):** Declarative configuration of Identity and Access Management (IAM) roles and policies under the principle of least privilege;
3. **Data Engineering:** Remote extraction and organisation into a medallion architecture alongside benchtop temporal interpolation;
4. **Predictive Modelling:** Training under strict temporal partitioning without data leakage and univariate feature safeguarding;
5. **OCI Containerisation and Serverless Deployment:** Docker packaging of the AWS Lambda function with global in-memory model persistence;
6. **NoSQL Persistence (DynamoDB):** Data modelling using composite keys (partition hash and temporal sort key) with strict numerical typing;
7. **Heterogeneous Ingestion (IoT Core):** Support for MQTT (QoS 1) and batched HTTPS REST endpoints paired with SQL routing rules;
8. **Auditing and Continuous Monitoring:** Verification of deterministic arithmetic parity ($\Delta < 10^{-4}$) and covariate shift detection via the two-tailed KS test.

3.1. Integrated Solution Topology

The cloud infrastructure was provisioned in the AWS us-east-1 region, leveraging an integrated ecosystem of

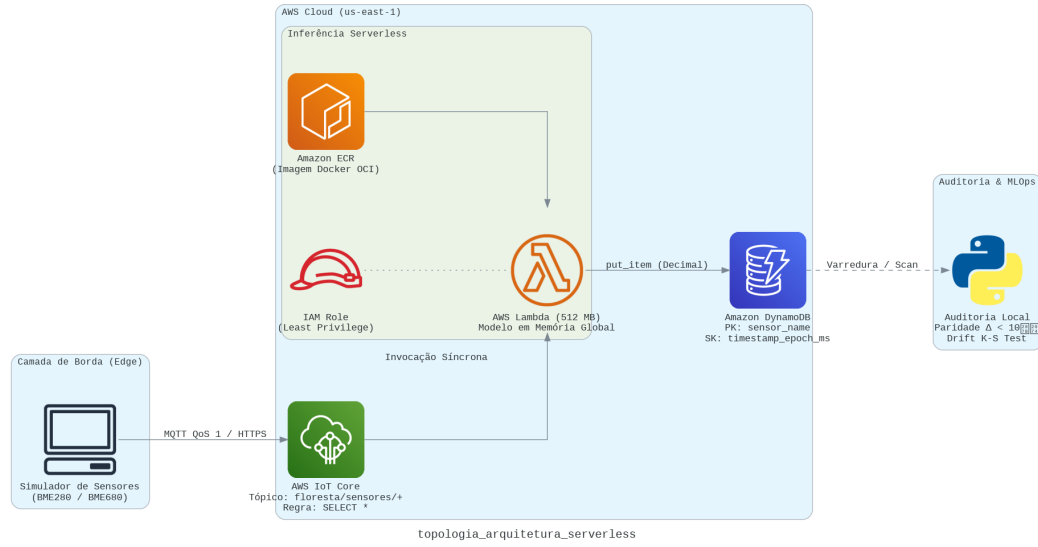


Figure 1: Integrated cloud architecture topology (AWS): heterogeneous ingestion via IoT Core, containerised serverless inference on AWS Lambda, atomic persistence in Amazon DynamoDB, and analytical audit modules.

managed services to establish a continuous, scalable, and loosely coupled data pipeline. Fig. 1 illustrates the resulting architectural topology, structured into sequential functional blocks spanning from edge telemetry emission to NoSQL persistence and analytical governance modules.

The architecture is organised into four interdependent operational zones:

- **Edge Layer:** Field devices emulating the hygrothermal behaviour of BME280 and BME680 transducers, configured to transmit data via MQTT over WebSockets (port 443 with SigV4 authentication) or through batch payloads via HTTPS REST;
- **Ingestion and Routing (AWS IoT Core):** A managed message broker configured to subscribe to the structured topic `floresta/sensores/+`. An SQL-based processing rule filters and routes telemetry payloads directly to the compute function without polling overhead;
- **Serverless Inference (AWS Lambda and ECR):** A microservice packaged under the OCI standard, instantiated from a private Amazon Elastic Container Registry (ECR) repository with an allocated operational memory limit of 512 MB. The calibration pipeline is cached in global execution memory to mitigate cold-start latencies, operating under an IAM role with strictly scoped permissions;

- **Persistence and Governance (DynamoDB and Local Auditing):** An on-demand provisioned NoSQL table receiving enriched records, from which local routines perform continuous queries to audit arithmetic parity and evaluate covariate shift.

3.2. Identity Governance and Privilege Isolation (IAM)

Operational access control across cloud components is governed by IAM, associating the execution of the serverless function with a dedicated role whose trust relationship is authenticated by the AWS Security Token Service (STS) via `lambda.amazonaws.com`. Fig. 2 documents the permission setup established during the pipeline validation phase within the AWS management console.

Policy name	Type	Attached en...
AmazonDynamoDBFullAccess_v2	AWS managed	1
AWSLambdaBasicExecutionRole-e...	Customer managed	1

Figure 2: Permission policies attached to the role.

For initial functional validation of the pipeline, the compute entity was bound to the custom `AWSLambdaBasicExecutionRole`

policy (enabling log stream creation and metric emission to Amazon CloudWatch Logs) and the managed AmazonDynamoDBFullAccess_v2 policy, ensuring uninterrupted persistence of inferences without transient permission blocks during stress testing.

Under the operational security guidelines established by the AWS Well-Architected Framework, granting broad access via pre-packaged managed policies introduces unnecessary attack surfaces in industrial environments, as it authorises administrative actions (e.g., table deletion and cross-table data access) under global wildcards ("Resource": "*"). To facilitate the transition to production under the principle of least privilege, a restricted declarative policy was structured in `utils/iam_dynamodb_policy.json`:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DynamoDBRestrito",
      "Effect": "Allow",
      "Action": [
        "dynamodb:PutItem",
        "dynamodb:UpdateItem",
        "dynamodb:GetItem"
      ],
      "Resource": "arn:aws:dynamodb:us-east-1:*:table/LeiturasSensoresCalibrados"
    }
  ]
}
```

This segregation strictly confines compute permissions to atomic write and query actions scoped to the Amazon Resource Name (ARN) of the project table, ensuring auditability and safeguarding against unauthorised access throughout continuous operation.

3.3. Data Engineering: Medallion Architecture and Temporal Interpolation

The pipeline begins with the extraction of historical time series collected by the low-cost nodes (*Floresta_08* to *Floresta_14*) and the benchtop reference station that is *Clima_01_EW*. The script `1.1-data_generator.py` establishes a secure SSH tunnel authenticated via an asymmetric RSA key to connect to the remote corporate database. Local persistence and processing follow the medallion architecture paradigm:

- **Bronze Layer (Raw Data):** Stores records exactly as extracted from the source, preserving the historical immutability of raw telemetry without filtering, type casting, or discarding physical anomalies;
- **Silver Layer (Cleaned and Synchronised Data):** Performs numerical schema conformity, purges records with corrupted timestamps, and conducts temporal synchronisation between the field sensor nodes and the reference station.

Because field sensors sample asynchronously, raw observations rarely coincide in the continuous time domain with reference benchtop measurements. During Silver Layer construction, linear temporal interpolation weighted by elapsed time is applied. Given an interval bounded by two consecutive

valid readings (t_a, x_a) and (t_b, x_b) , the interpolated value at intermediate timestamp $t \in [t_a, t_b]$ is computed as:

$$x(t) = x_a + \left(\frac{t - t_a}{t_b - t_a} \right) (x_b - x_a). \quad (8)$$

This processing reconstructs the continuous natural hygrothermal dynamics of the microclimate without introducing artificial step discontinuities, consolidating the unified database (`tabela_prata_dados_sensores.db`) that supports subsequent analytical stages.

3.4. Predictive Modelling and Data Leakage Mitigation

Once the Silver Layer was consolidated, the script `2.1-train_calibrator.py` executed the training routine for the dynamic calibrator. Due to the pronounced temporal autocorrelation inherent in micrometeorological variables, conventional stochastic data shuffling would induce data leakage, producing overfitting and deceptively optimistic performance estimates.

To safeguard the pipeline against this methodological bias, the temporal split was anchored strictly to the chronological series of the target variable provided by the benchtop reference station (*Clima_01_EW*). Based on the temporal ordering of this master station, two absolute chronological cutoff points ($t_{\text{split}, 1}$ and $t_{\text{split}, 2}$) were established, corresponding respectively to the 70th and 90th percentiles of its total temporal span. These universal cutoff timestamps were subsequently propagated across the entire unified database:

- **Training** ($t \leq t_{\text{split}, 1}$): Initial chronological window (representing the first 70% of the reference timeline) dedicated to learning decision tree split boundaries and fitting internal estimator parameters;
- **Testing** ($t_{\text{split}, 1} < t \leq t_{\text{split}, 2}$): Subsequent intermediate window (representing the next 20% of the reference timeline) utilised for hyperparameter tuning and model complexity control;
- **Validation** ($t > t_{\text{split}, 2}$): Final chronological horizon (the concluding 10% of the reference timeline) held strictly isolated for prospective out-of-time model evaluation.

Crucially, this partitioning strategy prioritised synchronous temporal integrity over balanced per-sensor record counts. Because edge nodes operate with asynchronous sampling rates and may experience intermittent field transmission dropouts, enforcing universal cutoff timestamps yielded sample proportions approximately close to, yet not identical to, the nominal 70/20/10 ratio across individual sensors. This marginal volumetric discrepancy reflects the empirical realities of physical WSN deployments and confirms the methodological fidelity of the pipeline, precluding any historical data contamination from future horizons.

The trained pipeline couples the `DecisionTreeRegressor` estimator with an upstream input feature imputation stage

(SimpleImputer). Whereas temporal interpolation in the Silver Layer established continuous temporal alignment for the reference target variable (y_{ref}), the SimpleImputer module operates exclusively on the predictor feature vector ($X = [H_{raw}, T, CO_2]$). Its inclusion within the serialised pipeline serves as a runtime operational failsafe: should a field node experience a transient sensor failure in a secondary covariate (such as an intermittent reading drop in the temperature or CO_2 channel) and transmit null values to AWS Lambda, the module imputes the missing feature without interrupting serverless handler execution.

Pipeline hyperparameters were tuned to mitigate overfitting and guarantee low-latency inference: within the SimpleImputer stage, a median imputation strategy was selected with dimensional retention enabled by the use of `keep_empty_features=True`; in the DecisionTreeRegressor, the MSE cost function (`squared_error`) was adopted, with maximum depth constrained to `max_depth=3`, minimum split samples set to `min_samples_split=10`, minimum leaf samples configured as `min_samples_leaf=5`, and the pseudo-random seed fixed to `random_state=42`.

Following temporal validation, the complete preprocessing and regression pipeline was packaged and serialised into the immutable artefact `best_model.joblib`, providing the static numerical asset required for subsequent OCI containerisation and serverless endpoint deployment.

3.5. Heterogeneous Ingestion in AWS IoT Core

Sensors deployed in forest environments face severe telecommunication instabilities caused by signal attenuation and multipath scattering across dense, humid foliage. To circumvent this constraint, the edge layer was engineered to support two complementary transmission regimes:

- **Continuous Event-Driven Transmission via MQTT:** Employs native MQTT over WebSockets via port 443, incorporating SigV4 signature-based authentication through the `awsiot-sdk` package. Each node publishes telemetry payloads to the topic `floresta/sensores/{sensor_name}` under a **QoS 1** (*At least once*) delivery guarantee. In this regime, the broker acknowledges receipt via a `PUBACK` packet, precluding telemetry losses during transient wireless link fluctuations;
- **Batch Ingestion via HTTPS REST (1.2):** Implemented using the `boto3` SDK, enabling devices subject to prolonged connectivity blackouts to offload accumulated historical buffers directly to the AWS IoT Core data API.

Cloud-side routing is governed by the AWS IoT Core rules engine, configured with the following SQL evaluation rule:

```
SELECT * FROM 'floresta/sensores/+'
```

This statement intercepts any payload published by the distributed microclimate nodes and synchronously invokes the serverless inference function.

3.6. OCI Containerisation and Serverless Runtime on AWS Lambda

To enable the deployment of the Scikit-Learn ecosystem without exceeding the 250 MB uncompressed size limit imposed by AWS Lambda on standard ZIP archives, the execution environment was packaged in compliance with the OCI standard.

The container image was derived from the official base distribution `public.ecr.aws/lambda/python:3.11` and pushed to a private Amazon ECR repository. Computational resources were allocated with **512 MB** of RAM and an execution timeout threshold of **15 s**.

To maximise runtime temporal efficiency, the function module `lambda_function.py` implements a global execution persistence pattern. Under the AWS Lambda execution lifecycle, variables instantiated outside the entrypoint handler persist within the memory space of the underlying Firecracker microVM across sequential invocations (warm state). Consequently, loading the pre-compiled model from disk is confined strictly to cold-start initialisation:

$$T_{exec} = T_{infer} + T_{db} \ll T_{boot} + T_{disk} + T_{infer} + T_{db} \quad (9)$$

The procedural execution flow of the serverless handler is formalised in Listing 1.

Listing 1: Serverless handler for real-time calibration on AWS Lambda.

```
import boto3
import joblib
from decimal import Decimal

# --- Escopo Global (Executado no Cold Start) ---
dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('LeiturasSensoresCalibrados')
model = joblib.load('best_model.joblib')

# --- Handler Serverless (Executado por Requisicao) ---
def lambda_handler(event, context):
    try:
        nome = event['sensor_name']
        timestamp = int(event['timestamp'])

        # Vetor de entrada: [umidade_bruta, temperatura, co2]
        x_input = [[
            event['umidade_bruta'],
            event['temperatura'],
            event['co2']
        ]]

        # Predicao deterministica em memoria
        y_pred = float(model.predict(x_input)[0])

        # Formatacao estrita para persistencia NoSQL
        item = {
            'sensor_name': nome,
            'timestamp_epoch_ms': timestamp,
            'umidade_bruta': Decimal(str(event['umidade_bruta'])),
            'umidade_calibrada': Decimal(str(round(y_pred, 4))),
            'temperatura': Decimal(str(event['temperatura'])),
            'co2': Decimal(str(event['co2']))
        }

        table.put_item(Item=item)
        return {'statusCode': 200, 'calibrado': y_pred}

    except Exception as e:
        return {'statusCode': 500, 'error': str(e)}
```

3.7. NoSQL Data Modelling and Persistence in Amazon DynamoDB

Storing high-frequency time series requires a persistence mechanism capable of accommodating fluctuating ingestion rates without input/output (I/O) latency degradation. Amazon DynamoDB, a fully managed NoSQL database, was selected and configured under the On-Demand capacity mode, thereby eliminating the need for upfront provisioning of Read Capacity Units (RCUs) and Write Capacity Units (WCUs).

The table `LeiturasSensoresCalibrados` was structured with a composite primary key, which is essential to optimise physical data partitioning:

- **Partition Key (PK):** Textual attribute `sensor_name`. DynamoDB submits this key to an internal cryptographic hash function, distributing write throughput uniformly across distinct physical storage partitions and mitigating the emergence of hot partitions;
- **Sort Key (SK):** Numerical attribute `timestamp_epoch_ms`, representing epoch time in milliseconds. This enables analytical queries to retrieve contiguous temporal slices per sensor node using range operators (*between*, *greater than*) with a retrieval time complexity of $\mathcal{O}(\log M + K)$, where M denotes the total item count within the partition and K represents the retrieved volume.

A critical data engineering nuance when interfacing DynamoDB through the boto3 SDK is its native incompatibility with standard Python floating-point types (`float`), whose inexact binary precision causes write-level schema rejections by the database engine. The serverless handler enforces strict type conversion of all floating-point quantities into the `decimal.Decimal` representation derived from string representations, ensuring numerical fidelity across all calibration decimal places. Table 1 summarises the schema attributes.

Fig. 3 confirms the continuous persistence of records, showing the coexistence of raw sensor emissions and the real-time calibrated values inferred by the serverless pipeline.

sensor_name (String)	timestamp_epoch_ms (Number)	absolute_pressure_hpa	co2_ppm	humidity_calibrated	humidity_raw	reading_time
Floresta_11	1788154291000	93.49	null	56.5982	50.75	2026-08-30 23...
Floresta_11	1788154303000	null	372	58.2897	59.85	2026-08-30 23...
Floresta_11	1788154314000	null	372	58.2897	59.79	2026-08-30 23...
Floresta_11	1788154324000	null	373	58.2897	59.8	2026-08-30 23...
Floresta_11	1788154341000	93.5	null	56.5982	50.74	2026-08-30 23...

Figure 3: Telemetry records in DynamoDB.

3.8. Auditing Strategy and Continuous Validation

Operational maintenance of the system under continuous execution relies on two decoupled analytical modules:

1. **Deterministic Parity Auditing:** Performs full scans across DynamoDB, retrieves the raw input tuples, and executes the local prediction artefact against identical

Table 1

Attribute Schema in the Amazon DynamoDB Table

Attribute	NoSQL Type	Pipeline Role
sensor_name	String (PK)	Physical node identification and partition hashing.
timestamp_epoch_ms	Number (SK)	Chronological ordering of the time series.
umidade_bruta	Number (Decimal)	Uncompensated raw sensor measurement.
umidade_calibrada	Number (Decimal)	Calibrated output generated by the regression model.
temperatura	Number (Decimal)	Auxiliary thermal covariate for compensation.
co2	Number (Decimal)	Gas concentration feature incorporated into the input vector.

vectors. The module compares the cloud-inferred values with the local outputs, certifying metrological compliance if the absolute deviation satisfies $|\hat{y}_{\text{cloud}} - \hat{y}_{\text{local}}| < 10^{-4}$;

2. **Covariate Shift Monitoring:** Assesses potential distortions in the statistical distributions of meteorological inputs by applying the two-tailed KS test across rolling telemetry windows, raising preventive retraining alerts before silent calibration degradation occurs in the field.

4. Results and Discussion

4.1. Overall Metrological Performance

The corrective capability of the dynamic calibrator was evaluated across the continuous temporal validation dataset, comprising 18,550 synchronised samples from nodes *Floresta_08* to *Floresta_14* evaluated against the *Clima_01_EW* reference benchtop station. The evaluated window spans a weekend period characterised by clear-sky meteorological conditions, providing an operational benchmark to assess pipeline behaviour across stable diurnal and nocturnal cycles under low environmental variance at the master station. In such regimes, dimensionless metrics such as R^2 lose analytical expressiveness, rendering MAE and RMSE the definitive benchmarks of metrological compliance.

The uncompensated raw field baseline exhibited substantial systematic bias, yielding a MAE of 5.404% RH and a RMSE of 6.382% RH. Table 2 summarises the comparative performance across the evaluated architectural topologies.

The shallow Decision Tree with constrained tree depth ($d = 3$) emerged as the champion architecture, achieving

Table 2
Comparative Model Performance on the Validation Dataset

Model	MAE (%)	RMSE (%)	MAE Gain
Raw Sensor (Baseline)	5.404	6.382	—
Global Linear Regression	4.725	4.773	12.56%
Global Ridge Regression	4.725	4.773	12.56%
Ridge with Per-Sensor Offset	4.491	4.555	16.89%
Random Forest (30 trees, $d = 3$)	2.510	2.689	53.55%
Decision Tree ($d = 3$)	2.422	2.580	55.19%

Table 3
Metrological Performance Disaggregated by Physical Sensor (Champion Model)

Sensor	Readings	Raw MAE	Calib. MAE	Δ MAE (%)
<i>Floresta_08</i>	1695	3.885%	3.820%	+1.67%
<i>Floresta_09</i>	1719	6.498%	3.527%	+45.72%
<i>Floresta_10</i>	1730	5.246%	4.535%	+13.55%
<i>Floresta_11</i>	1729	7.392%	4.325%	+41.49%
<i>Floresta_12</i>	882	4.704%	1.015%	+78.42%
<i>Floresta_13</i>	847	5.061%	5.281%	-4.34%
<i>Floresta_14</i>	851	2.242%	5.286%	-135.77%

* Positive Δ MAE values indicate error reduction.

a net absolute error reduction of 55.19% (yielding a final MAE of 2.422% RH). Strictly linear formulations achieved marginal improvements ($\approx 12.56\%$), underscoring their inability to capture the non-linear hygrothermal transfer characteristics of capacitive transducers. Although the Random Forest ensemble yielded substantial improvements (53.55%), the shallow single tree outperformed its global predictive accuracy while delivering crucial engineering advantages: negligible memory allocation and sub-millisecond execution latencies within the Firecracker microVM.

Table 3 details the node-level response of the champion model broken down by physical sensor.

The granular disaggregation presented in Table 3 demonstrates that the largest corrective gains were concentrated in nodes exhibiting pronounced baseline systematic drift: node *Floresta_12* achieved a 78.42% reduction in absolute error (reaching an operational MAE of 1.015% RH), whereas *Floresta_09* and *Floresta_11* demonstrated average error reductions exceeding 40%. The volumetric record divergence between sensor groups stems from distinct firmware sampling cadences configured during initial benchtop testing.

Conversely, the global calibration model penalised node *Floresta_14*, whose uncompensated baseline was already operating in close proximity to the reference instrument during the window (initial MAE of 2.242% RH). This localised performance drop is an artefact of the loss function converging towards the global cluster mean residuals across a window with narrow physical excitation amplitude (stable weekend solar irradiance), which restricts exposure across broader hydrodynamic regimes. To mitigate this effect without burdening the cloud MLOps pipeline with numerous segregated model artefacts, subsequent iterations

will incorporate a two-stage hybrid topology: macro non-linear inference orchestrated centrally in AWS Lambda coupled with lightweight residual gain-and-offset parameter vectors embedded directly within the edge firmware of each transducer.

4.2. Residual Analysis and Identity Scatter Characteristics

The stochastic homogeneity of the prediction errors and the validity of the metrological correction were investigated using a 1:1 identity scatter plot alongside the empirical probability density of residuals ($e_i = y_{\text{ref},i} - \hat{y}_i$). The diagnostic evaluation processed $N = 18,550$ paired observations collected during the temporal validation window, benchmarking field node measurements (*Floresta_08* to *Floresta_14*) against the *Clima_01_EW* reference benchtop station. The consolidated diagnostic panels are depicted in Fig. 4.

In Fig. 4(a), the dispersion of uncompensated raw readings (shown in red) demonstrates a pronounced positive systematic bias, consistent with the baseline MAE of 5.404% RH. Following compensation by the serverless pipeline (shown in blue), the data points converge tightly along the ideal 1:1 bisector line ($y = x$), reducing the MAE to 2.422% RH—yielding a net 55.19% gain in dimensional compliance relative to the benchtop reference.

Fig. 4(b) illustrates the error distribution using a residual histogram overlaid with a Kernel Density Estimation (KDE) curve. The residuals exhibit an essentially unimodal distribution with narrow dispersion ($\sigma = 1.3720\%$ RH) centered at a shifted mean of $\mu = +2.3779\%$ RH. This remaining positive mean offset is a direct consequence of the structural regularization imposed by the shallow tree architecture ($d = 3$), which prioritises low-latency inference and generalisation robustness over complete bias elimination across homogenous microclimates. The pronounced contraction in residual variance ($\sigma < 1.4\%$) demonstrates that the serverless calibrator successfully suppresses the erratic non-linear dispersion of the capacitive transducers, restoring statistical predictability to the edge sensor nodes.

4.3. Verification of Deterministic Numerical Parity

A critical vulnerability in production MLOps pipelines involves the silent loss of numerical parity across environments: models developed and validated locally can produce divergent outputs during cloud inference due to microarchitectural instruction discrepancies or variations across compiled C/C++ linear algebra runtimes (such as BLAS and LAPACK).

The auditing module 4.1-auditoria_pipeline.py conducted an automated reconciliation over the telemetry records persisted in Amazon DynamoDB ($N = 70$). For each record written by the serverless Lambda handler, the module parsed the raw input vector $\mathbf{x}_k = [H_{\text{raw}}, T, \text{CO}_2]$, re-executed local inference using the baseline `best_model.joblib` artefact, and computed the absolute scalar divergence:

$$\Delta_k = |\hat{y}_{\text{cloud}}(k) - \hat{y}_{\text{local}}(k)|, \quad \forall k \in [1, N]. \quad (10)$$

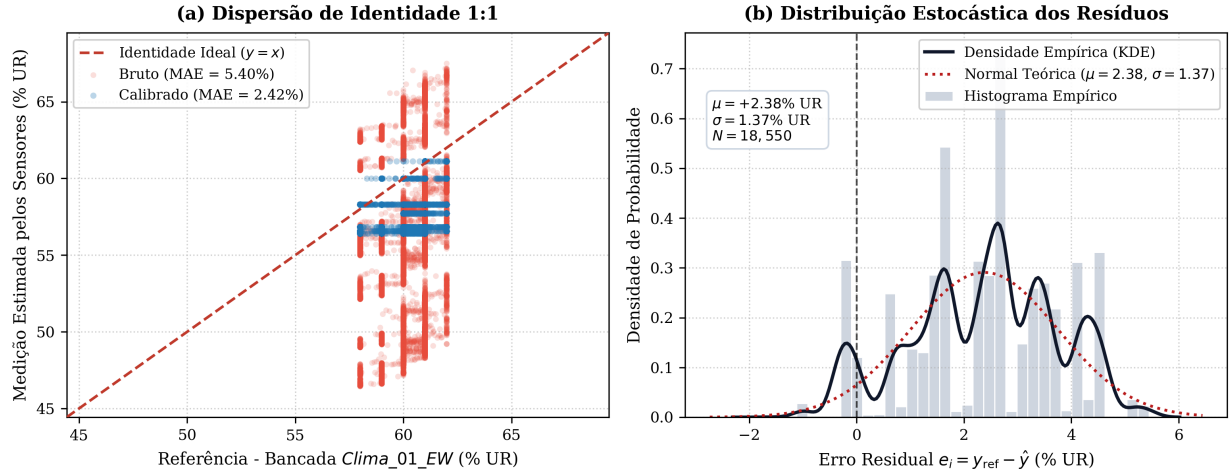


Figure 4: Diagnostic calibration validation ($N = 18,550$): (a) 1:1 identity scatter plot comparing raw and calibrated readings against the master reference station; (b) Empirical Kernel Density Estimation (KDE) curve and residual histogram contrasted with the theoretical normal distribution.

```

=====
RELATÓRIO DE AUDITORIA: DYNAMODB (AWS LAMBDA) VS. PIPELINE LOCAL
=====
sensor_name      reading_time      humidity_raw      humidity_calibrated      pred_local      divergencia_nuvem_local
Floresta_08 2026-08-30 23:58:18+00:00      57.09      58.2897      58.289710      0.000010
Floresta_08 2026-08-30 23:58:28+00:00      57.11      58.2897      58.289710      0.000010
Floresta_08 2026-08-30 23:58:38+00:00      57.11      58.2897      58.289710      0.000010
Floresta_08 2026-08-30 23:58:48+00:00      57.11      58.2897      58.289710      0.000010
Floresta_08 2026-08-30 23:58:58+00:00      57.15      58.2897      58.289710      0.000010
Floresta_08 2026-08-30 23:59:08+00:00      57.16      58.2897      58.289710      0.000010
Floresta_08 2026-08-30 23:59:20+00:00      60.15      58.2897      58.289710      0.000010
Floresta_08 2026-08-30 23:59:31+00:00      60.11      58.2897      58.289710      0.000010
Floresta_08 2026-08-30 23:59:42+00:00      60.08      58.2897      58.289710      0.000010
Floresta_08 2026-08-30 23:59:58+00:00      57.11      58.2897      58.289710      0.000010
Floresta_09 2026-08-30 23:58:31+00:00      64.38      59.9857      59.985687      0.000013
Floresta_09 2026-08-30 23:58:42+00:00      64.40      59.9857      59.985687      0.000013
Floresta_09 2026-08-30 23:58:48+00:00      51.30      56.3982      56.398241      0.000041
Floresta_09 2026-08-30 23:58:58+00:00      51.27      56.3982      56.398241      0.000041
Floresta_09 2026-08-30 23:59:08+00:00      51.28      56.3982      56.398241      0.000041
=====
Divergência Máxima Absoluta: 0.000041%
STATUS: PARIDADE CONFIRMADA (100% determinístico e livre de discrepâncias numéricas).

```

Figure 5: Automated parity audit report benchmarking AWS Lambda predictions against local offline inference, validating exact deterministic compliance ($\max \Delta_k = 0.000041\%$).

The empirical evaluation registered a maximum absolute disparity of:

$$\max_{1 \leq k \leq N} \Delta_k = 0.000041\% < 10^{-4}. \quad (11)$$

This residual deviation remains strictly below the designated metrological tolerance threshold ($\tau = 10^{-4}\%$ RH). It stems entirely from the four-decimal truncation applied prior to NoSQL persistence ($\text{round}(\hat{y}, 4)$), confirming that OCI-compliant containerisation ensures full numerical determinism across offline model development and live serverless inference. Fig. 5 displays the diagnostic report generated by the parity auditing terminal.

4.4. Preventive Covariate Shift Detection and Production Readiness

In industrial MLOps pipelines, sustained inferential integrity requires continuous observability mechanisms to mitigate model degradation stemming from physical sensor ageing, particulate deposition across porous transducer membranes, or seasonal climate transitions. To evaluate the production readiness of the architecture, the monitoring module 4.2-drift_monitoring.py was deployed to operate as an automated statistical sentinel, intercepting silent distributional shifts before uncalibrated errors propagate into downstream ecophysiological models

The sensitivity of this governance mechanism was assessed by applying the two-sample KS test to the empirical CDF of the raw relative humidity series for node *Floresta_11*. The calibration baseline (initial 70% training partition) was

Table 4

Experimental Covariate Shift Diagnostics and Metrological Degradation (*Floresta_11*)

Operational Window	D_{KS}	p-value	MAE (%)	Governance Action
Training (Baseline Ref.)	0.0000	1.000	0.941	Nominal Operation
Window 1 (Night Transition)	0.4885	$< 10^{-4}$	2.879	Alert / Drift Detected
Window 2 (Diurnal Peak)	0.4024	$< 10^{-4}$	2.027	Alert / Drift Detected
Window 3 (Closing Window)	0.4717	$< 10^{-4}$	4.328	Alert / Drift Detected

benchmarked against three consecutive operational monitoring windows within the prospective validation horizon. Table 4 summarises the trajectory of the D_{KS} distance statistic, the corresponding p -values, and the associated metrological impact on MAE.

During Window 1 (nocturnal transition), the diurnal hydrothermal inversion substantially modified the statistical distribution of input readings, driving the inter-distribution distance to $D_{KS} = 0.4885$ alongside a decisive rejection of the null hypothesis of stochastic equivalence ($p < 10^{-4}$). In accordance with statistical learning theory under non-stationary conditions, this input distribution shift directly correlated with downstream metrological degradation, culminating in an elevated MAE of 4.328% RH in Window 3. Fig. 6 illustrates the divergence between the empirical CDFs.

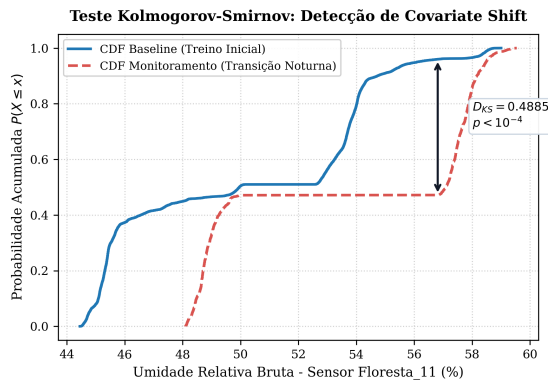


Figure 6: Divergence in CDFs under the KS test.

Crucially, the drift alerts and resulting retraining recommendations recorded during this evaluation reflect the constrained temporal span of the initial experimental baseline. Because the model was fitted across a compact observation window, the training baseline lacked sufficient statistical support across the full range of the circadian cycle. Consequently, the transition into nocturnal conditions was treated as an unobserved regime by the estimator, inflating the D_{KS} statistic and increasing residual error. This behaviour is expected when operating over restricted initial baselines. In continuous production deployment, ingesting extended

Table 5

Production Serverless Execution Profile (AWS Lambda)

Operational Metric	Observed Value (CloudWatch)
Effective Per-Invocation Latency (Warm State)	10.95 ms
Billed Duration	11.00 ms
Provisioned RAM	512.0 MB
Peak Memory Allocation	232.0 MB (45.3%)
Operational Headroom	280.0 MB free
Execution Error Rate / HTTP Failures	0.000%

longitudinal datasets that capture multi-day circadian and seasonal dynamics will establish a comprehensive reference distribution, suppressing these premature triggers. Thus, during prototype validation, the module functions primarily as an architectural diagnostic tool demonstrating production readiness, verifying that the serverless pipeline reliably intercepts real-time covariate shifts prior to automating retraining loops across the complete operational lifecycle.

4.5. Serverless Infrastructure Execution Profile

The computational efficiency and latency of the containerised serverless pipeline were audited using production telemetry recorded in Amazon CloudWatch Logs. Table 5 summarises the operational benchmarks recorded during the processing of telemetry payloads transmitted by the sensor nodes.

The mean observed latency of 10.95 ms per invocation demonstrates the lightweight nature of the analytical pipeline. Because deserialising the `best_model.joblib` artefact and instantiating the Boto3 resource client occur once within the global scope during initialisation (cold start), subsequent warm-state invocations perform only vector unpacking, decision-tree inference, and the atomic write to DynamoDB with sub-millisecond execution times.

Memory consumption stabilised at 232 MB, corresponding to 45.3% of the provisioned 512 MB ceiling. This 280 MB operational headroom safeguards the microservice against transient heap allocation surges triggered by socket network buffering or payload parsing expansion, ensuring sustained, deterministic execution across the environmental monitoring network.

5. Conclusion

This paper presented the design, implementation, experimental validation, and production auditing of an end-to-end data engineering and MLOps pipeline for the dynamic calibration of low-cost environmental sensor networks. The investigation systematically addressed cross-environment coupling, systematic transducer bias, and latency constraints within an event-driven serverless cloud architecture.

The principal contributions and findings of this study are summarised as follows:

1. **Demonstrated Metrological Efficacy:** The shallow decision-tree calibrator ($d = 3$) reduced the MAE from

5.404% RH to 2.422% RH across an unbroken out-of-time evaluation set of 18,550 synchronised readings benchmarked against the reference master station. This corresponds to a net 55.19% improvement in dimensional accuracy, alongside a marked contraction of the residual error variance ($\sigma = 1.3720\%$ RH);

2. **Numerical Determinism and Runtime Parity:** Packaging the pipeline into an immutable OCI container preserved IEEE 754 floating-point consistency across offline development and cloud environments, constraining maximum inference divergence to $\max \Delta_k = 0.000041\%$ RH (strictly below the $10^{-4}\%$ RH metrological threshold and attributable entirely to decimal truncation prior to NoSQL persistence);
3. **Drift Monitoring and Production Readiness:** Implementing the two-tailed KS test confirmed high sensitivity to diurnal distributional shifts ($D_{KS} = 0.4885$), validating the architectural capability of the pipeline to intercept input disturbances in real time and lay the empirical groundwork for automated retraining triggers;
4. **Serverless Computational Efficiency:** Production telemetry from AWS Lambda demonstrated an average warm-state execution latency of 10.95 ms and peak RAM utilisation of 232 MB (45.3% of the provisioned 512 MB allocation), confirming substantial operational headroom for low-cadence forest telemetry streams;
5. **Operational Security and Least Privilege:** Enforcing granular IAM policies decoupled access control and secured atomic writes to DynamoDB without throughput degradation or concurrency bottlenecks.

Direct extensions of this work include: (i) deploying a two-stage hybrid calibration scheme that pairs macro non-linear inference in AWS Lambda with a lightweight residual offset compensation vector embedded directly within node firmware at the edge; (ii) expanding the reference temporal baseline across multi-season cycles to provide dense statistical support for automated retraining loops orchestrated via AWS Step Functions and Amazon EventBridge; and (iii) quantising and converting the analytical artefact for autonomous, near-sensor execution on ultra-low-power edge microcontrollers.

Acknowledgements and Declaration of Generative AI Usage

The author acknowledges the use of the Gemini large language model (Google) as an assistive tool for drafting and stylistic refinement of this manuscript. The conception of the physical experiments, the acquisition and validation of operational cloud telemetry, and the initial textual draft were carried out entirely by the author. The artificial intelligence model was employed exclusively to assist with structural outlining, English language polishing, and $\text{\LaTeX}$ formatting. All generated output was subjected to rigorous verification and critical review by the author, who assumes full technical and intellectual responsibility for the published work.

References

- [1] Banks, A., Gupta, R., 2014. MQTT Version 3.1.1. OASIS Standard. OASIS Standard.
- [2] Bergmeir, C., Benítez, J.M., 2012. On the use of cross-validation for time series predictor evaluation. *Information Sciences* 191, 192–213.
- [3] Massey Jr, F.J., 1951. The kolmogorov-smirnov test for goodness of fit. *Journal of the American statistical Association* 46, 68–78.
- [4] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., et al., 2011. Scikit-learn: Machine learning in python. *the Journal of machine Learning research* 12, 2825–2830.
- [5] Petersen, J., Kristensen, J., Elarga, H., Andersen, R.K., Midtstraum, A., 2018. Accuracy and air temperature dependency of commercial low-cost ndir co2 sensors: An experimental investigation, in: 4th International Conference On Building Energy & Environment, pp. 203–207.
- [6] Rango, A.D., Furnari, L., Cortale, F., Senatore, A., Mendicino, G., 2025. Wildfire early warning system based on a smart co2 sensors network. *Sensors* 25, 2012.
- [7] scikit-learn developers, 2024a. 3.3. metrics and scoring: quantifying the quality of predictions — mean absolute error. https://scikit-learn.org/stable/modules/model_evaluation.html. Accessed em: 4 set. 2026.
- [8] scikit-learn developers, 2024b. Decision trees — scikit-learn documentation. <https://scikit-learn.org/stable/modules/tree.html>. Accessed em: 4 set. 2024.
- [9] Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.F., Dennison, D., 2015. Hidden technical debt in machine learning systems. *Advances in neural information processing systems* 28.
- [10] Spinelle, L., Gerboles, M., Villani, M.G., Aleixandre, M., Bonavita-cola, F., 2017. Field calibration of a cluster of low-cost commercially available sensors for air quality monitoring. part b: No, co and co2. *Sensors and Actuators B: Chemical* 238, 706–715.
- [11] Taştan, M., 2025. Machine learning-based calibration and performance evaluation of low-cost internet of things air quality sensors. *Sensors* 25, 3183.
- [12] Varghese, B., Buyya, R., 2018. Next generation cloud computing: New trends and research directions. *Future generation computer systems* 79, 849–861.
- [13] Wu, M.F., Zhong, M.Z., Tsai, H.Y., Tseng, Y.S., Wen, C.Y., 2025. Edge-docker-based architecture for intelligent indoor air quality management with sensing calibration and automatic controlling. *Indoor Air* 2025, 1031975.
- [14] Zaidan, M.A., Motlagh, N.H., Fung, P.L., Lu, D., Timonen, H., Kuula, J., Niemi, J.V., Tarkoma, S., Petäjä, T., Kulmala, M., 2020. Intelligent calibration and virtual sensing for integrated low-cost air quality sensors. *IEEE Sensors Journal* 20, 13638–13652.